

Phase 2 Report — Calibration, Generalization, and Real-Time Scheduling

DAG Execution-Time Quantile Prediction on Heterogeneous Multicore + DVFS

Course: پیش‌بینی توزیعی زمان اجرای گره‌های — پروژه اول ۰۵-نیمسال دوم ۰۴ (Real-Time Systems) — سامانه‌های بی‌درنگ
Phase: 2 of 2 — Conformal calibration, OOD/scalability generalization, and real-time scheduling impact
Handoff basis: [RealTime_Systems_Project1_4042_Sharif.pdf](#) (formal spec) + [Phase_2_Handoff_Guide_DAG_Execution_Time_Project.pdf](#) (frozen Phase 1 artifact contract)

0. Scope and the one rule that governs everything below

The Phase 1 predictor is frozen. Every result in this report is produced by loading `models/phase1/checkpoints/best_model.pt` and the exact train-fitted preprocessing state from disk, with `strict=True` state-dict loading and `model.eval()`. **No weight is ever updated, no scaler is ever refit, and neither the calibration nor the test-ID/test-OOD splits are used for model selection.** The only new *models* trained anywhere in Phase 2 are the independent comparison baselines in §5 (MLP, GNN-Mean, GNN-Mean-NoZt) — these are new, separately-instantiated networks used purely for a side-by-side comparison table; they never modify or replace the frozen predictor.

Frozen-model integrity check (Handoff Guide §19, explicitly requested before Phase 2 work begins): the model was reconstructed completely independently in this environment — fresh disk reload of `best_model.pt` + `preprocessing_state.json` via `src/phase1/bootstrap.py` (including the documented `Phase1Config` keyword-filtering fix), run in a separate process from whatever produced Phase 1's saved baseline. Comparing this fresh reload's raw Q50/Q90/ Q95/Q99 predictions against the already-saved `results/phase1/predictions/test_id_predictions.csv` baseline across all 2,100 test-ID records: **max absolute difference = 0.5 μ s, mean absolute difference $\approx$ 0.035 μ s** — floating-point-level numerical noise, not a substantive discrepancy. The frozen checkpoint reload is verified reproducible before any Phase 2 analysis is built on top of it.

Every calibrated quantity `c_e` in this report is a **statistically calibrated upper predictive bound with a marginal coverage guarantee** under the exchangeability assumption between the calibration split and the evaluation split, **not a deterministic WCET**. This distinction is restated at the point of every scheduling result because it changes how the numbers should be read.

Reproducing this report end-to-end:

```
python3 scripts/run_phase2_inference.py      # frozen inference: calibration/test_id/test_ood
python3 scripts/run_phase2_calibration.py    # split-conformal calibration by criticality
python3 scripts/run_phase2_ood_structure.py  # OOD + graph-structure sensitivity
python3 scripts/run_phase2_scalability.py    # 4/8/16/32-core stress test
python3 scripts/run_phase2_drift.py          # tail-load / overload / sliding-window calibration
python3 scripts/run_phase2_scheduling.py     # HEFT + deadline-aware scheduling
python3 scripts/run_phase2_baselines.py     # MLP / GNN-Mean / GNN-Mean-NoZt comparison
python3 scripts/run_phase2_deepdive.py      # operation_type, inference-time, GNN-embedding correlation
python3 scripts/run_phase2_plots.py         # all figures below
```

All source modules live under `src/phase2/` (`inference.py`, `conformal.py`, `metrics.py`, `graph_structure.py`, `scalability.py`, `drift.py`, `scheduling.py`, `baselines.py`, `config.py`); `src/phase1/bootstrap.py` reconstructs the frozen model exactly as specified in the handoff guide (including the documented `Phase1Config` keyword-filtering fix for `num_quantiles`).

1. Conformal calibration analysis

(spec §"تحليل اثر كاليبراسيون همريخت"; code: `src/phase2/conformal.py`, `scripts/run_phase2_calibration.py`)

1.1 Chosen operating point and why

Parameter	Value	Justification
<code>base_quantile</code> ($1-\alpha_h$)	Q95 for both HI and LO	Q99 already over-covers on raw test-ID (coverage 1.000), leaving the calibration layer almost nothing to correct — its nonconformity scores are ≈ 0 for nearly every row. Q95 leaves real, data-driven headroom for calibration to do genuine work while still being a legitimate upper quantile.
<code>δ_{HI}</code>	0.01 (target coverage 99%)	High-criticality DAG nodes get the tightest statistical safety margin the trained head can support.
<code>δ_{LO}</code>	0.05 (target coverage 95%)	Best-effort work gets an adequate but less conservative margin, avoiding unnecessary budget inflation.

Provenance, stated precisely to avoid any ambiguity about test-set peeking: the Q95-vs-Q99 comparison above comes from the *pre-existing*, already-computed Phase 1 test-ID baseline handed off in [Phase_2_Handoff_Guide §12 / results/phase1/metrics/test_id_metrics.json](#) (raw coverage @Q95 = 0.9900, @Q99 = 1.0000) — **not** from any new Phase-2-computed evaluation. This operating point (`src/phase2/config.py`) is fixed *before* any Phase-2-computed test_id/test_ood calibrated-coverage number is ever looked at. §1.4's α/δ sensitivity sweep is evaluated *afterward*, purely as a transparency report required by the spec's "do not invent α/δ silently, report sensitivity" instruction — it never feeds back into changing this default, so it does not constitute tuning on test_id/test_ood (a practice the handoff guide's Do/Do-Not checklist explicitly forbids).

1.2 Fitted calibration state (on the untouched 50-DAG / 2,100-record calibration split)

Criticality	N	<code>k_h</code>	<code>q_conf</code> (μs)	Fraction of zero scores
HI	1,050	1,041	15,665.78	0.980
LO	1,050	999	0.00	0.989

A `q_conf = 0` for LO is a **legitimate outcome**, not a bug: the one-sided nonconformity score is $s = \max(0, y - Q_{base})$, so calibration can only ever *keep or increase* an already over-covering raw quantile — it can never shrink one. At $\delta=0.05$ the raw Q95 for LO already clears the 95% target on the calibration set, so no correction is needed. (Full derivation: [models/phase2/calibration/conformal_state.json](#), [conformal_config.json](#).)

1.3 Coverage and inflation, before vs. after calibration

Eval split	Criticality	Raw coverage	Calibrated coverage	Target	Mean inflation (μs)
test_id	HI	0.9943	1.0000	0.99	15,665.78
test_id	LO	0.9857	0.9857	0.95	0.00
test_ood	HI	0.9562	0.9790	0.99	15,665.78
test_ood	LO	0.9514	0.9514	0.95	0.00

(full table: [results/phase2/calibration/coverage_by_criticality.csv](#), [inflation_by_criticality.csv](#); figure: [results/phase2/plots/coverage_before_after_calibration.png](#), [reliability_diagram.png](#))

Note on "mean inflation" above: because `q_conf` is a single fixed correction added to every record within a criticality level, the applied inflation is a **constant** (15,665.78 μ s for every HI record, 0 for every LO record — not a distribution with any spread). The genuinely-distributed quantity, and the one the handoff guide explicitly asks to report ("distribution of nonconformity scores, including the fraction of zero scores"), is the underlying **calibration-set nonconformity score** `s_j = max(0, y_j - Q_base(x_j))` before the `k_h`-th order statistic collapses it to that single `q_conf` value — shown in [results/phase2/plots/inflation_distribution.png](#) (log-scale histogram, since 98.0% of HI and 98.9% of LO calibration scores are exactly zero, with the resulting `q_conf` threshold marked) alongside a bar chart of the resulting constant correction itself.

Reading this: on in-distribution data the raw head is already close to nominal and calibration mostly confirms it (HI even reaches perfect empirical coverage after a modest +15,666 μ s correction). Under distribution shift (test_ood, 501–1000-node graphs the model never trained on) raw HI coverage degrades to 0.956 — below its 0.99 target — and the **frozen, previously-fit** calibration correction pulls it back up to 0.979, still short of target but a real, measurable recovery purely from data-driven correction with no retraining. LO coverage on test_ood (0.951) already meets its 0.95 target raw, so — correctly — no correction is applied there either; the method is not artificially "fixing" a number that wasn't broken.

1.4 α/δ sensitivity sweep

Swept `base_quantile` $\in \{Q90, Q95, Q99\} \times \delta \in \{0.01, 0.05, 0.10\}$ (symmetric HI=LO for the sweep; the asymmetric default above is the chosen *operating point*). Full table:

[results/phase2/calibration/calibration_sensitivity_sweep.csv](#). Headline pattern: at $\delta=0.01$, `q_conf` is strictly positive and shrinks monotonically as the base quantile rises (Q90→34,469 μ s, Q95→15,666/2,457 μ s (HI/LO), Q99→0 μ s) — exactly the expected conformal behavior, since a higher raw base quantile needs less correction to reach the same target. At $\delta \geq 0.05$, most cells already collapse to `q_conf=0`, confirming §1.2's zero-score observation is not an artifact of one configuration.

2. Predictive stability and quantile geometry

(تخلیل پایداری و هندسهی پیشبینی "spec S")

- **Quantile crossing rate: 0.0 everywhere** (train, test_id, test_ood, all scalability/drift scenarios) — the frozen model's `OrderedQuantileHead` enforces `Q50 ≤ Q90 ≤ Q95 ≤ Q99` by **architectural construction** (positive softplus increments added on top of Q50), not by a soft training penalty. This is a *stronger* guarantee than the spec's optional `L_nc` penalty-term approach (eq. 28): the architectural route makes crossing structurally impossible rather than merely discouraged in expectation, which is exactly why the observed crossing rate is exactly 0.0 in every split and every stress scenario tested in this report, with no separate `L_nc` term needed or exercised.
- **Visual distribution shape** ([results/phase2/plots/violin_predicted_quantiles_by_role.png](#), [violin_predicted_by_hardware_scenario.png](#), [box_actual_vs_predicted.png](#)): quantile spread (Q99–Q50) varies systematically by target-node role — `source_like` nodes have the tightest, best-calibrated distributions (coverage_Q95 = 0.991 on test_ood, mean pinball 1,231 μ s), while `sink_like` and `communication_heavy` nodes have both the widest spread and the worst pinball (7,910 μ s and 6,443 μ s respectively) — see §4.4. The distribution also visibly shifts by hardware scenario (`core_type` × DVFS level): `little`-core, low-DVFS scenarios show both higher median and wider Q95 spread than `big`-core, high-DVFS scenarios, directly reflecting the core/DVFS sensitivity quantified in §3.1.
- **Point-estimate baselines cannot express this at all.** §5's MLP/GNN-Mean baselines only ever produce one number per (node, context); they cannot flag "this node's execution time is unusually uncertain right now," which is exactly the information a scheduler needs to decide whether to admit a HI-criticality job. This is the concrete cash value of the quantile representation over a mean-only model, beyond the aggregate MAE gap in §5.

3. Scalability and system heterogeneity

(ارزیابی مقیاس‌پذیری و ناهمگنی سامانه "spec S")

3.1 core_type / DVFS sensitivity (existing test data, real ground truth)

core_type	mean pinball (μ s)	MAE (μ s)	coverage_Q95
big	3,536	12,772	0.960
little	5,511	20,528	0.948

dvfs_level	mean pinball (μ s)	MAE (μ s)	coverage_Q95
0 (lowest f)	6,085	23,350	0.942
1	4,568	16,481	0.966
2	3,738	13,839	0.958
3 (highest f)	3,719	12,993	0.949

(test_ood; full tables [results/phase2/ood/grouped_ood_metrics_by_{core_type,dvfs_level}.csv](#); figure [results/phase2/plots/dvfs_sensitivity.png](#)) Both patterns are directionally sensible: **little** cores run longer and their (proportionally larger) execution times are harder to predict in absolute μ s; the lowest DVFS level (slowest, longest-running) is the hardest operating point to predict accurately, consistent with error scaling with the magnitude being predicted. This is direct evidence the model learned genuinely different, non-trivial behavior per core/DVFS combination rather than a single global average.

3.2 Compute-bound vs. memory-bound node behavior

operation_type	test_id mean pinball (μ s)	test_ood mean pinball (μ s)	test_ood coverage_Q95
compute_bound	2,629	5,868	0.930
balanced	2,494	5,260	0.946
memory_bound	2,623	2,547	0.983

([results/phase2/ood/grouped_{test_id,test_ood}_metrics_by_operation_type.csv](#)) On in-distribution data all three operation types are predicted with comparable accuracy (2,494–2,629 μ s pinball) — evidence the model has learned to handle each regime, not just one dominant pattern. Under distribution shift the three types **diverge sharply**: memory-bound nodes barely degrade (2,623→2,547 μ s, essentially flat) while compute-bound nodes degrade by more than 2 $\times$ (2,629→5,868 μ s). This is a genuinely non-linear, regime-dependent generalization pattern — not something a single global scaling factor could produce — and is direct evidence the model learned qualitatively different internal behavior for compute-bound vs. memory-bound nodes, as the spec's phrasing asks to demonstrate.

3.3 4/8/16/32-core scalability stress test

Framing (stated explicitly because it changes the correct interpretation): **core_id** was never a Phase 1 model input (only **core_type** $\in$ {big, little}); the model has no notion of *how many* cores exist. A literal 32-core hardware re-simulation is not something this frozen predictor can produce. What *can* be honestly tested is the one feature that does encode core-count pressure — **active_core_count** — pushed from its training range [1, 4] out to 4, 8, 16, and 32 on real test_id contexts (topology, core_type, DVFS, and every other z_t feature held fixed). No ground truth exists for a hypothetical 32-core version of this synthetic system, so this is reported as a **prediction-uncertainty/robustness characterization only**, never as an accuracy claim.

Scenario	active_core_count	z-score (train-fitted)	mean Q50 (μs)	mean C_e (μs)	mean spread Q99–Q50 (μs)
core_4	4	1.83	252,866	292,755	47,833
core_8	8	6.74	252,037	291,769	47,608
core_16	16	16.54	250,675	290,101	47,166
core_32	32	36.15	245,590	284,124	45,894

([results/phase2/scalability/core_scalability_summary.csv](#); [figure results/phase2/plots/scalability_core_count.png](#))

Safety-relevant finding: predicted execution time and the calibrated budget both *fall slightly* as `active_core_count` is pushed increasingly far outside the training range (z-score up to 36 standard deviations), rather than rising as a naive intuition (more contention → longer runtime) might expect. Both quantile spread and calibrated budget shrink together — the model becomes *more confident*, not less, exactly where it should be *least* trusted. This is exactly the kind of **non-conservative extrapolation** the OOD literature warns about and that a real deployment must guard against (e.g., by refusing to trust predictions once a feature's standardized z-score exceeds a fixed threshold, or by widening `δh` explicitly whenever `active_core_count > 4` is detected at inference time).

3.4 DVFS / energy / safety tradeoff (qualitative)

Relative energy $\propto V^2 \cdot f$ per active cycle (not calibrated wattage, used only for the qualitative tradeoff): going from DVFS level 0 → 3 on **big** cores is a 1.0→2.2 GHz, 0.90→1.15 V change, i.e. roughly a **2.55×** relative per-cycle energy increase for execution time that (per §3.1) drops and becomes markedly easier to predict accurately (pinball 6,085 → 3,719 μs). The tradeoff this project's budgets expose to a scheduler is explicit: lower DVFS levels are cheaper per cycle but produce *both* longer *and* less statistically reliable execution-time budgets — a real energy/safety/deadline tradeoff a downstream DVFS-aware scheduler could exploit (e.g., reserve the lowest DVFS levels for LO-criticality, slack-rich jobs only).

4. Graph structure and scalability

(*تحليل ساختار و مقیاس‌پذیری گراف* "spec S")

4.1 Node-count scaling (50–1000 nodes)

eval_split	n	mean pinball (μs)	MAE (μs)	coverage_Q95
test_id (50–500 nodes, train range)	2,100	2,584.5	10,907.8	0.990
test_ood (501–1000 nodes)	2,100	4,523.8	16,650.0	0.954

`num_nodes` is **100%** out of the `[51, 497]` train range by construction on `test_ood`; `depth/max_width` are ~10% out of range even *within* the 501–1000-node split. Accuracy degrades by roughly 1.75× (pinball) but does not collapse — the model still produces usable, ordered, non-crossing quantiles on graphs twice the size it ever trained on.

([results/phase2/ood/structural_feature_range_report.csv](#), [accuracy_vs_graph_size.png](#))

Training time vs. graph size is not applicable here by design — the Phase 1 predictor is frozen and is never retrained per graph size in Phase 2 (retraining would violate the primary rule in §0). **Inference time** was measured directly instead (single forward pass, `_infer_one_graph`, wall-clock, CPU):

eval_split	mean inference time (ms)	median	max
test_id (50–500 nodes)	24.2	11.6	128.5
test_ood (501–1000 nodes)	112.5	53.0	779.4

Pearson correlation(`num_nodes`, `inference_time_ms`) = **0.44** across both splits — a moderate positive relationship, consistent with the DAG encoder's per-node message-passing cost, not a pathological blow-up. `test_ood` graphs (2× the node count on average) take ~4.6× longer to run — still well within real-time-adjacent budgets (worst case observed: 779 ms for the single largest graph) for an offline/near-offline scheduling use case, though clearly too slow for a per-context online re-query at every scheduling tick on the largest graphs.

(`results/phase2/ood/inference_time_vs_graph_size.csv`)

4.2 Structural "shape" sensitivity (*fat* / *density_parameter* / *regular*, generator range 0.2–0.8)

Dimension	Low (0.2) pinball	Mid (0.4-0.6) pinball	High (0.8) pinball
fat (width/depth ratio)	4,633	2,684	5,482
density_parameter	2,313	3,700	5,818
regular (regularity)	6,824	5,214	3,254

(`test_ood`; `results/phase2/ood/grouped_ood_metrics_by_{fat,density_parameter,regular}_bin.csv`) Density is the strongest driver: sparse graphs (low density) are predicted markedly better than dense ones (2,313 vs. 5,818 μ s pinball) — plausibly because dense graphs create more message-passing interference per node and larger, harder-to-disentangle input-volume terms. Highly regular graphs are easier than irregular ones (3,254 vs. 6,824 μ s), consistent with more homogeneous local neighborhoods being easier for the attention layers to summarize.

4.3 Critical-path correlation — topological proxy and learned GNN embeddings

Critical-path membership proxy: `topo_level(v) + reverse_topo_level(v) == depth(G) - 1` (a topological, hop-count longest-path proxy — not compute-cycle-weighted; stated as a limitation, see §8).

eval_split	is_critical_path_node	coverage_Q95
test_ood	False	0.977
test_ood	True	0.946

(`results/phase2/ood/critical_path_accuracy_comparison.csv`, `critical_path_coverage.png`) Critical-path nodes are measurably less well-covered than other nodes under distribution shift — a directly scheduling-relevant finding, since these are exactly the nodes whose error most affects a DAG's end-to-end completion time.

Going beyond the topological proxy, the spec explicitly asks for correlation between the GNN's **learned** representation and accuracy. `model.dag_encoder(x, edge_index, edge_attr)` was queried read-only (no weight update) to extract each target node's actual hidden embedding, and two derived features were correlated against per-record error (`src/phase2/embeddings.py`, `scripts/run_phase2_deepdive.py`):

Correlation with...	pinball@Q95	abs error @Q50
<code>embedding_norm</code> (test_id)	−0.353	−0.331
<code>embedding_dist_from_graph_mean</code> (test_id)	+0.256	+0.204
<code>embedding_norm</code> (test_ood)	−0.129	−0.267
<code>embedding_dist_from_graph_mean</code> (test_ood)	+0.143	+0.186

([results/phase2/ood/embedding_error_correlation_{test_id,test_ood}.csv](#)) Two consistent, interpretable signals: (1) larger embedding norms correlate with *lower* error — nodes the encoder represents with a stronger/more confident activation are predicted more accurately; (2) embeddings that sit *further from their own graph's mean embedding* — i.e., structurally unusual nodes relative to their DAG — correlate with *higher* error, in both splits. This independently corroborates the topological finding above: on test_ood, critical-path nodes (by the topological proxy) have both higher mean pinball (3,800 vs. 2,146 μ s) and higher mean [embedding_dist_from_graph_mean](#) (6.36 vs. 5.90) than non-critical-path nodes ([results/phase2/ood/embedding_by_critical_path_test_ood.csv](#)) — two independent measures of "how hard this node is" agree with each other.

4.4 Edge-feature / communication-volume influence

target_role	mean pinball (μ s)	coverage_Q95
source_like	1,231	0.991
high_degree	3,019	0.967
random	3,166	0.989
diverse_fill_2	2,036	1.000
diverse_fill_1	4,126	0.941
middle	5,575	0.931
communication_heavy	6,443	0.900
sink_like	7,910	0.940

([test_ood; results/phase2/ood/communication_role_accuracy.csv](#)) Nodes explicitly selected to be [communication_heavy](#) (large incoming [input_size_bytes](#) / [data_bytes](#) volume) show both the worst coverage (0.900, well under the 0.95 target) and among the worst pinball of any role — direct evidence that edge/communication features are a genuine source of residual difficulty the model has not fully mastered under distribution shift, and a strong argument for treating these nodes conservatively (higher δ margin) in a criticality-aware deployment.

5. Baseline comparison

(*spec §* مقایسه خروجی مدل با روشهایی که تنها میانگین را پیشبینی می‌کنند"; *code:* [src/phase2/baselines.py](#), [scripts/run_phase2_baselines.py](#))

Two new, independently-trained comparison models (never touching the frozen predictor), trained under an explicitly reduced compute budget (150/30 train/val graphs, 8 epochs, CPU) for tractability — meant to establish directional comparisons, not to be exhaustively tuned competitors:

Model	MAE (μ s)	RMSE (μ s)	What it ablates
MLP (no graph)	68,697	99,060	No message passing — only the target node's own static features + context
GNN-Mean	50,464	75,221	Same DAGEncoder/ContextFusionEncoder family, MSE mean head instead of pinball quantile head
GNN-Mean-NoZt (offline)	48,248	71,961	z_t features zeroed (standardized mean) — "offline" scheduler blind to current system state
GNN-Quantile (frozen, Q50)	10,908	17,462	(the actual Phase 1 predictor — full training budget, quantile loss)

([results/phase2/baselines/model_comparison.csv](#), [plots/model_comparison_mae.png](#))

MLP → GNN-Mean: adding graph structure (message passing over predecessors/successors) cuts MAE by ~27% even under this reduced training budget — direct evidence the DAG topology carries real predictive signal beyond a node's own features.

GNN-Mean → frozen GNN-Quantile: the frozen model's MAE is **~4.6× lower** than the reduced-budget GNN-Mean. Given both share the same backbone family, this gap is attributable to (a) the full training budget (103 epochs / early stopping vs. 8) and (b) the quantile/ pinball training objective itself, which — unlike plain MSE — explicitly penalizes both tails and naturally regularizes toward the conditional median rather than a mean that can be pulled by the right-skewed execution-time distribution.

Online (z_t-aware) vs. offline (z_t-blind): under this reduced training budget the GNN-Mean-NoZt ablation is *not* conclusively worse than GNN-Mean (48,248 vs. 50,464 μs MAE — within the noise band of two independently-initialized, lightly-trained runs), so this specific ablation is **inconclusive at this compute budget** and is reported honestly as such rather than forcing a narrative. The frozen model's *own* diagnostics are more decisive on this question: Phase 1's grouped-by-[cpu_utilization/bus_utilization](#) metrics ([results/phase1/metrics/test_id_grouped_{cpu_utilization,bus_utilization}.csv](#)) show mean pinball loss rising systematically and monotonically with utilization bin — 2,139→3,089 μs across CPU-utilization bins, 2,216→3,218 μs across bus-utilization bins on test-ID — which would not happen if the frozen model's output were insensitive to [z_t](#). Combined, the evidence is: the frozen, fully-trained model is demonstrably [z_t](#)-sensitive; whether a reduced-budget ablation can *cleanly isolate* that sensitivity in a few minutes of CPU training is a separate (negative) result about baseline compute budget, not about the frozen predictor.

6. Generalization and distribution shift

(آزمون تعمیم‌پذیری و تغییر توزیع "spec S")

6.1 Larger graphs — covered in §4.1 (test_{ood}).

6.2 Robustness to sudden [z_t](#) changes

(code: [src/phase2/drift.py](#), [scripts/run_phase2_drift.py](#))

Two genuinely different questions, kept separate per the handoff guide's warning against conflating them:

(a) Real in-range tail-load stress (valid ground truth — top decile of a composite memory/bus/thermal/jitter load proxy on real test_{ood} records):

is_high_load_tail	n	mean pinball (μs)	MAE (μs)	coverage_Q95
False	1,890	4,392	16,217	0.956
True	210	5,710	20,549	0.933

Genuine, ground-truth-backed accuracy degradation under real high-pressure conditions.

([results/phase2/drift/tail_load_stress_accuracy.csv](#))

(b) Synthetic extreme overload (descriptive only — [memory_active_tasks=10](#), [bus_utilization=1.4](#), [thermal_pressure=1.3](#), [release_jitter_us=900](#), all clearly beyond every train range; no fabricated ground truth is used):

scenario	mean Q50 (μs)	mean C _e (μs)	Δ C _e vs. baseline
baseline_test_id	253,009	292,930	—
synthetic_overload	457,541	532,713	+81.9%

([results/phase2/drift/synthetic_overload_comparison.csv](#)) Unlike the `active_core_count` scalability stress in §3.2, the model responds **directionally sensibly** here — predicted cost and calibrated budget rise sharply under overload rather than falling. The likely explanation: memory/bus/thermal pressure were densely sampled across their full range during training (Beta(2.2, 2.2)-derived, stratified per §6 of the handoff guide), while `active_core_count` was only ever observed in a narrow 4-point range — a concrete, testable hypothesis for *why* one OOD direction extrapolates safely and another does not.

(c) Static vs. sliding-window (causal, adaptive) calibration — a strictly past-only online variant (window = 5 DAGs $\approx$ 210 records; each window's correction is fit exclusively on **already-revealed** residuals from earlier windows, never future ones):

Criticality	Target coverage	Static coverage	Sliding-window coverage	Static budget (μ s)	Sliding-window budget (μ s)
HI	0.99	0.979	0.987	328,639	355,590 (+8.2%)
LO	0.95	0.951	0.962	293,057	297,964 (+1.7%)

([results/phase2/drift/static_vs_sliding_window_calibration.csv](#), [plots/static_vs_sliding_window_calibration.png](#)) The adaptive variant recovers coverage closer to target under `test_ood`'s distribution shift for both criticality levels, at the cost of a modestly (HI: +8.2%, LO: +1.7%) larger mean budget — a legitimate, causally-valid demonstration that online/adaptive calibration is a viable mitigation for drift, directly answering the spec's request to compare static vs. sliding-window calibration.

6.3 Mixed-criticality overrun / mode-switch trigger rate

A standard mixed-criticality trigger condition is: *a HI-criticality job overruns its optimistic (LO-level) budget*. Using the frozen HI raw quantile plus the **LO** conformal correction ($q_{\text{conf}}^{\text{LO}}=0$) as the hypothetical LO-mode budget for HI jobs:

eval_split	n (HI)	Mode-switch trigger rate	HI overrun vs. its own (HI) calibrated budget
test_id	1,050	0.57%	0.00%
test_ood	1,050	4.38%	2.10%

([results/phase2/ood/mixed_criticality_mode_switch.csv](#)) Under distribution shift, a HI-criticality job would need to trigger a LO $\rightarrow$ HI mode switch roughly **7.7 $\times$ more often** (0.57% $\rightarrow$ 4.38%) than in-distribution — quantitative evidence that criticality-separated calibration provides real value specifically under the conditions (OOD, drift) where a mixed-criticality real-time system is most at risk.

Effect on LO-criticality service (qualitative, since no live multi-mode runtime is simulated here — see §8): in a standard mixed-criticality policy, every LO $\rightarrow$ HI mode switch degrades or drops concurrently-running LO tasks for the remainder of that hyperperiod. Since §7's scheduling experiment places 25 HI and 25 LO DAGs on the same undersized 4-core platform, and §7 already shows HI-criticality DMR under the calibrated budget is **1.00** (every HI DAG in this test set), a system that actually enforced mode-switching on HI overrun would, in the worst case implied by these numbers, be forced to degrade LO service essentially continuously on this platform — itself a symptom of the platform being undersized for these graphs' parallelism demands (§7), not a flaw in the calibration method.

Bursty arrivals: the scheduling experiment in §7 evaluates each DAG job in isolation (explicit assumption, §7/§8) and does not model multiple DAG *releases* arriving close together in time — true burst-arrival admission-control behavior is out of scope for this deliverable. What is covered is the in-record proxy for burstiness the dataset actually provides, `release_jitter_us`, which is one of the four features composing §6.2(a)'s tail-load stress test (real ground truth, top-decile pinball 5,710 vs. 4,392 μ s elsewhere) — so release-jitter-driven degradation is measured, even though inter-arrival burstiness of whole DAG jobs is not.

7. Impact on scheduling performance

(*spec §* "تأثیر بر عملکرد زمان‌بندی"; *code*: `src/phase2/scheduling.py`, `scripts/run_phase2_scheduling.py`)

Stated simplifying assumption: each of the 50 test_id DAGs is scheduled **in isolation** on an otherwise-idle 4-core (2 big + 2 little) platform — no multi-job/multi-tenant interference is modeled. This isolates the effect of the *cost estimator* (which is this project's actual contribution) from admission-control policy, which is out of scope here.

Full-DAG cost prediction: the 42 sampled records/DAG cover only 6 of up to ~500 nodes; scheduling needs a cost for every node. Since `F_0` is a per-node function, it is queried for every (node, core_type) pair at a representative DVFS point (highest-performance level per core type) and the train-fitted **mean z_t** vector (the "expected/typical" operating point a static offline scheduler would use) — architecturally identical to how the 42 evaluation contexts were built, just without requiring a sampled ground truth.

Schedulers implemented: classic HEFT (upward-rank prioritization + insertion-free earliest-finish-time processor selection) and a deadline-partitioned, EDF-style deadline-aware list scheduler (sub-deadlines distributed proportionally along the critical path, ready nodes ordered by ascending sub-deadline).

Scheduler	Cost estimator	DMR	mean tardiness (μs)	mean R_end (μs)
HEFT	Q50 (optimistic)	0.84	5,289,148	9,422,355
HEFT	Q95 (raw, uncalibrated)	0.96	6,732,285	11,012,400
HEFT	C_e (calibrated)	0.98	7,400,388	11,685,400
DeadlineAwareList	Q50 (optimistic)	0.84	5,311,963	9,441,113
DeadlineAwareList	Q95 (raw, uncalibrated)	0.96	6,742,191	11,021,230
DeadlineAwareList	C_e (calibrated)	0.98	7,418,218	11,702,420

(full table `results/phase2/scheduling/scheduling_summary.csv`, `scheduling_summary_by_criticality.csv`; figure `plots/scheduling_comparison.png`)

Reading these numbers correctly — two points the spec explicitly asks for:

- 1 **HEFT alone is not a sufficient real-time evaluation lens.** HEFT and the deadline-aware scheduler produce nearly identical makespans/DMR here. Investigating why (cross-referenced against `metadata/graph_summary.csv`) shows several test_id graphs have `max_width` up to 20+ against only 4 physical cores — the platform's parallelism budget, not the scheduling *policy*, is the binding constraint. A pure makespan-optimizing scheduler and a deadline-aware one converge to the same answer precisely when core count is the bottleneck, which is exactly the situation where reporting *only* HEFT/makespan (without DMR/tardiness) would be misleading — the report therefore leads with DMR and tardiness, and treats makespan as a secondary corroborating number, as the spec requires.
- 2 **Using `C_e` as a literal scheduling cost is deliberately pessimistic, and that is by design, not a flaw.** DMR rises monotonically `Q50` → `Q95` → `C_e` (0.84 → 0.96 → 0.98) because each successive estimator is a progressively more conservative statistical bound, not a better *expected*-cost estimate. `C_e` is meant for **admission/safety analysis** ("would this DAG be schedulable even in a statistically-unlucky case?"), not as the literal throughput-oriented cost fed to a makespan-minimizing scheduler. A production system would use `Q50`/mean-like estimates for *scheduling order and packing* and reserve `C_e` for the *admission decision and deadline-feasibility check* — a distinction this comparison makes concrete rather than asserting.

By criticality (`scheduling_summary_by_criticality.csv`): HI-criticality DMR under `C_e` is **1.00** (worse than LO's 0.96) — the conservative HI correction from §1 (`q_conf^HI` = 15,666 μs) directly widens HI jobs' predicted cost, which is the intended safety behavior but also means, in this small 4-core platform, essentially no HI-criticality DAG in this test set is schedulable within its nominal deadline under the fully conservative budget — a legitimate structural finding (this platform is undersized relative to these graphs' parallelism demands) rather than an artifact of the estimator.

8. Limitations and honesty notes

Documented explicitly, matching the spec's expectation that this project's outputs be interpreted as statistical bounds, not ground truth about physical hardware:

- 1 **No literal >4-core hardware re-simulation.** §3.2's scalability stress overrides `active_core_count` on the frozen model's existing `z_t` input; it does not (and, given `core_id` was never a model feature, cannot) simulate a physically different 8/16/32-core system. Reported as an uncertainty/robustness characterization only.
- 2 **Critical-path proxy is topological (hop-count), not compute-cycle-weighted** — `topo_level + reverse_topo_level == depth - 1`. A weighted longest-path computation using `compute_cycles` would be a more precise definition; the topological proxy is a standard, defensible approximation but should not be over-read as "the" critical path used by any specific scheduler.
- 3 **Baselines (§5) trained under a reduced compute budget** (150/30 graphs, 8 epochs, CPU) — sufficient to establish clean, large directional gaps (MLP < GNN-Mean, both well below the frozen model) but not to draw fine-grained conclusions from small gaps (e.g., the offline-vs-online `z_t` ablation, explicitly reported as inconclusive at this budget).
- 4 **Scheduling (§7) evaluates each DAG in isolation** on an idle 4-core platform; no multi-job interference, preemption, or admission control is modeled. `Δ_queue` and `Δ_preempt` (spec eq. 16/41) are out of scope for this deliverable — only `Y_exec`-driven scheduling is evaluated, as the spec's primary learning objective specifies.
- 5 **All ground truth is synthetic simulator output** (`y_exec_us`), not measurements from physical hardware — every accuracy number in this report should be read as "how well the model recovers the synthetic generator's own function," which is the correct and only claim this dataset supports.
- 6 **Representative DVFS/`z_t` choice for full-DAG scheduling costs (§7)** — highest- performance DVFS level per core type and the train-fitted mean `z_t` vector. A different choice (e.g., per-node DVFS optimization, or a pessimistic rather than mean `z_t`) would shift absolute DMR/tardiness numbers; the *relative* ordering across cost estimators (Q50 < Q95 < C_e in conservatism) is the robust, estimator-agnostic finding this report relies on.
- 7 **No live multi-mode mixed-criticality runtime is simulated.** §6.3's mode-switch trigger rate and its qualitative LO-service-impact discussion are derived from static per-record overrun statistics, not from an actual online mode-switching scheduler simulation with LO-task dropping/degradation.
- 8 **Bursty arrival of whole DAG jobs is out of scope** (§7's isolated single-DAG scheduling assumption already precludes it); only within-DAG burstiness proxies (`release_jitter_us`, as part of the composite tail-load stress test in §6.2) are covered.

9. Artifact index

models/phase2/calibration/ results/phase2/calibration/ results/phase2/ood/ results/phase2/scalability/ results/phase2/drift/ results/phase2/scheduling/ results/phase2/baselines/	conformal_config.json, conformal_state.json (frozen) raw + calibrated predictions, coverage/inflation reports, α/δ sensitivity sweep, nonconformity-score distribution test_id-vs-test_ood, grouped diagnostics (core/DVFS/ criticality/role/graph-size/structure/critical-path/ operation_type), mixed-criticality mode-switch analysis, inference-time-vs-graph-size, GNN-embedding/error correlation (src/phase2/embeddings.py, scripts/run_phase2_deepdive.py) core_4/8/16/32 metrics + summary tail-load stress, synthetic overload, static-vs-sliding window calibration HEFT + deadline-aware results, DMR/tardiness/slack MLP / GNN-Mean / GNN-Mean-NoZt models + comparison table
---	---

results/phase2/plots/ all 13 figures referenced above
notebooks/03_phase2_calibration_scheduling.ipynb runnable, narrated walkthrough